

Chapter 6

Tracking and Identifying Program Symbols

The previous chapters gave us three critical language implementation skills. We can now write parsers for input sentences, we can make parsers build abstract syntax trees, and we can build tree walkers that visit or rewrite those trees. That means we're almost ready to tackle *semantic analysis*. Semantic analysis is just a fancy way to say we're "sniffing a program to see whether it makes sense." The term *semantics* is synonymous with "meaning."

We'll explore semantic analysis in Chapter 8, *Enforcing Static Typing Rules*, on page 196, but we need to build some infrastructure before we get there. To enforce language semantics, we need to track symbol definitions and be able to identify those symbols later. (A symbol is just a name for a program entity like a variable or method.) It's like keeping track of characters in a novel. Novels introduce characters and then refer to them later. To understand the story, we've got to remember each character and something about their attributes. Killing off a character means there should be no further references to it. Similarly, computer language sentences can define and reference symbols in code blocks. At the end of the code block, the symbols go out of scope.

Language applications track symbols in an abstract data structure called a *symbol table*.

In this chapter, we will define and implement two basic symbol table patterns:

- Pattern 16, *Symbol Table for Monolithic Scope*, on page 156. All symbols exist within a single scope (set of symbols). Simple property files and early BASIC are the best examples.
- Pattern 17, *Symbol Table for Nested Scopes*, on page 161. There are multiple scopes and scopes can nest inside other scopes. C is a typical language that has nested scopes. Scopes start and end at the start and end of language structures such as functions.

Each pattern includes sample code that shows how to populate and query the symbol table. To get the most out of these patterns, we need to learn about representing program symbols, grouping symbols into scopes, and resolving symbols to their definitions.

6.1 Collecting Information About Program Entities

To build a symbol table, we need to formalize what we do implicitly when we read and write software. First, let's figure out how to represent program entities. Take a look at the following C++ code:

```
class T { ... }; // define class T
T f() { ... }    // define function f returning type T
int x;          // define variable x of type int
```

We unconsciously define three *symbols* (program entities) in our head: class T, function f, and variable x. To build a language application, we need to mimic this in software. For those definitions, we need to do something like the following:

```
Type c = new ClassSymbol("T");           // define class
MethodSymbol m = new MethodSymbol("f", c); // define method
Type intType = new BuiltInTypeSymbol("int"); // define int type
VariableSymbol v = new VariableSymbol("x", intType); // define var x
```

Those constructors pretty much tell us what we need to know for each symbol. But, to be explicit, each of those symbols has at least the following three key properties:

- *Name*: Symbols are usually identifiers like x, f, and T, but they can be operators too. For example, in Ruby, we can use operators like + as method names.
- *Category*: That is, what kind of thing the symbol is. Is it a class, method, variable, label, and so on. To validate method call f(), for

example, we need to know that `f` is a method, not a variable or class.

- *Type*: To validate operation $x+y$, for example, we need to know the types of x and y . Dynamically typed languages like Python track type information at run-time. Statically typed languages like C++ and Java track type information at compile time. The programmer usually has to explicitly identify each symbol's type (in some languages, the compiler infers it).

We distinguish program entities via these three parameters. For example, function `f` that returns a string is obviously different from a variable of type integer called `x`. In a novel, we might have an elf character (the category) called Sri (the name) that becomes a soldier (the type).

A symbol table implements each symbol category with a separate class, holding the name and type as properties. We can factor out those shared properties into a common Symbol superclass:

```
public class Symbol {
    public String name; // All symbols at least have a name
    public Type type;   // Symbols have types
}
```

VariableSymbol is the simplest program entity category and looks like this:

```
public class VariableSymbol extends Symbol {
    public VariableSymbol(String name, Type type) { super(name, type); }
}
```

Let's turn our attention now to user-defined types like classes and **structs**. For consistency, let's represent user-defined types like any other program symbol. We can derive BuiltInTypeSymbol and ClassSymbol from Symbol (though they don't need the type field).

To distinguish between user-defined types and other program symbols, it's a good idea to tag types with a Type interface. For example, here's the class that represents built-in types like `int` and `float`:

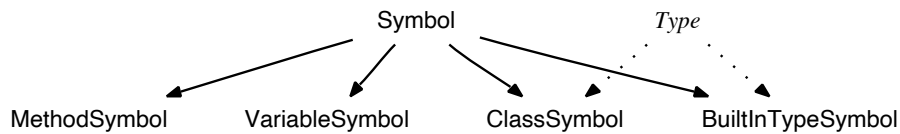
```
public class BuiltInTypeSymbol extends Symbol implements Type {
    public BuiltInTypeSymbol(String name) { super(name); }
}
```

There's not much to interface Type because we're using it only as a tag:

```
public interface Type { public String getName(); }
```

I like to think of interfaces as roles that classes can play. In this case, symbols implementing Type can play the role of a type. Here is a typical

(partial) class hierarchy showing the categories subclassing `Symbol` and types implementing `Type`:



Representing symbols in our symbol table is not too bad. Things only get “interesting” when we have to track those symbols and look them up again.

6.2 Grouping Symbols into Scopes

A scope is a code region with a well-defined boundary that groups symbol definitions (in a dictionary associated with that code region). For example, class scopes group members; function scopes group parameters and local variables. Scope boundaries usually coincide with begin and end tokens such as curly braces (sort of like the “hello” and “good-bye” of a phone conversation). We call this *lexical scoping* because the extent of a scope is lexically delimited. Perhaps a better term is *static scoping* because we can track scopes just by looking at the source code; that is, without executing it. Here’s a list of scope characteristics that often differ between languages:

- *Static vs. dynamic scoping*: Most languages have static scoping, but some (like classic LISP and PostScript) have dynamic scoping. Think of dynamic scoping as allowing methods to see the local variables of invoking methods.
- *Named scopes*: Many scopes, like classes and methods have names, but global and local scopes don’t.
- *Nesting*: Languages usually allow some form of scope nesting. For example, nested code blocks enclosed in curly braces often introduce new scopes. C++ and Java allow nested classes. Languages typically limit nesting to certain combinations. Python allows function definitions within other functions, but Java does not.
- *Contents*: Some scopes allow declarations, some allow statements, and some allow both. C **structs** only allow declarations. Python’s global space allows declarations and executable code.

- *Visibility*: The symbols in a scope might or might not be visible to some other code section. The fields in a C **struct** are visible to any code section. The fields in a class have visibility modifiers such as **public** and **private** that indicate which code sections can reference them. Local variables typically aren't visible to other functions.

To represent a scope, we'll use an interface so that we can tag entities like functions and classes as scopes. For example, a function is a kind of Symbol that also plays the role of a scope. Scopes have pointers to their enclosing scopes (we'll talk about this more later) and can have names. Scopes don't need to track the code region from which we create them. Instead, the AST for the code regions point to their scopes. This makes sense because we're going to look up symbols in scopes according to what we find in the AST nodes.

For now, we only care about the first three methods in interface Scope (we'll learn about `resolve()` in the next section):

```
public interface Scope {
    public String getScopeName();           // do I have a name?
    public Scope getEnclosingScope();       // am I nested in another?
    public void define(Symbol sym);         // define sym in this scope
    public Symbol resolve(String name);     // look up name in scope
}
```

To get the ball rolling, let's see what we can do with a single Scope object.

Monolithic Scopes

Early programming languages such as BASIC had a single global scope. Today, only simple languages like configuration files have a single scope. For example, here's a property file:

```
host=antlr.org           # define properties in single, global scope
port=80                  # a set can act as a symbol table
webmaster=parrrt@antlr.org
```

Within a scope, a symbol can only represent a single entity. That means that redefining a property either overwrites the previous value or is an error. Tracking symbols for monolithic scope just means maintaining a set of Symbol objects. As we encounter definitions, we create symbols and add them to the set. Later, we'll look them up again by name. A dictionary that maps symbol names to Symbol objects works best for this. Pattern 16, *Symbol Table for Monolithic Scope*, on page 156 defines the symbol table we'd need to handle languages with a single scope.

Multiple and Nested Scopes

Having multiple scopes lets us reuse the same name in different code regions to identify different program entities. For example, variable name `x` in the following methods refers to two different variables:

```
void f() { int x;   printf(x); } // this x lives in f's scope
void g() { float x; printf(x); } // this x lives in g's scope
```

To avoid ambiguity, programming languages use context to figure out which symbol we're talking about. Context corresponds to the scope surrounding the symbol and any enclosing scopes. Within the context of method `f()`, the `x` argument of `printf()` resolves to the `int` local variable. In the context of `g()`, `x` resolves to the `float` variable. At the close of each method, the local variables go out of scope (become invisible).

Programming languages also let us nest one scope within another. Nesting a scope is like allowing someone to pop into your office and ask a question while you're already in a meeting. We return to the meeting after answering the question. To track nested scopes, we push and pop scopes onto a scope stack. As we encounter a new scope, we push it on the scope stack. The top of the stack is called the *current scope*. As we exit a scope, we pop it off the stack, revealing the previous scope as the current scope.

All symbol definitions occur within the current scope. For example, here is some C++ code commented with scope information:

```
❶ // start of global scope
  int x;           // define variable x in global scope
❷ void f() {       // define function f in global scope
    int y;         // define variable y in local scope of f
❸   { int i; }     // define variable i in nested local scope
❹   { int j; }     // define variable j in another nested local scope
  }
❺ void g() {       // define function g in global scope
    int i;         // define variable i in local scope of g
  }
```

The numbered icons identify the various scopes. The following diagram shows what the scope stack looks like after the definition of each new symbol. The stack grows and shrinks over time (growing upward) as we push and pop scopes.

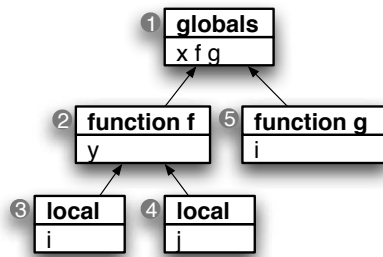
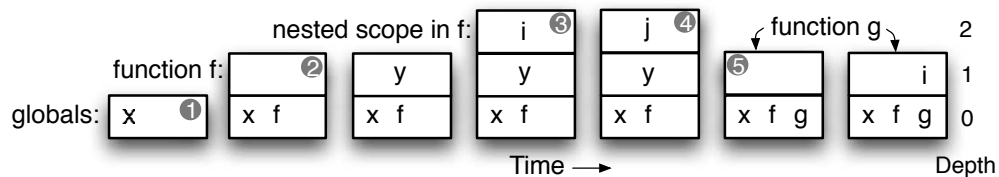


Figure 6.1: Scope tree for global, function, and local scopes



By popping a scope off the scope stack, we’re effectively throwing it away. In many cases, though, we’re going to need that scope later to check sentences for correctness, generate output, and so on. To keep everything around, we want a data structure called a *scope tree* (a tree of Scope’s) that can function like a collection of stacks. In Figure 6.1, we see the scope tree for the previous C++ code. Every path from a node to the scope tree root represents a stack of scopes. For example, node 4 has implicit scope stack 421, and node 5 has implicit scope stack 51. The levels of the tree correspond to the scope nesting levels. As we move up a level in the tree, we move out a level in the program scopes. Instead of pushing a scope onto a stack, we’re going to add a child to a scope tree.

The tree might look a little funny because the nodes point at their parents instead of the reverse. In practice, though, we’ll look up symbols by scanning upward toward the root (see the next section).

Building a scope tree boils down to executing a sequence of these operations: push, pop, and def. All of the patterns in this chapter and the next populate symbol tables using these core abstract operations.

- push. At the start of a scope, push a new scope on the scope stack. This works even for complicated scopes like classes. Because we are building scope trees, push is more like an “add child” tree

construction operation than a conventional stack push. To make things more concrete, here's an implementation preview:

```
// create new scope whose enclosing scope is the current scope
currentScope = new LocalScope(currentScope);    // push new scope
```

- pop. At the end of a scope, pop the current scope off the stack, revealing the previous scope as the current scope. pop moves the current scope pointer up one level in the tree:

```
currentScope = currentScope.getEnclosingScope(); // pop scope
```

- def. Define a symbol in the current scope. We'll always define symbols like this:

```
Symbol s = «some-new-symbol»;
currentScope.define(s); // define s in current scope
```

Let's look at the sequence of operations we'd need a parser to perform in order to build the scope tree in Figure 6.1, on the previous page. For now, don't worry about what triggers these actions (the pattern implementations go over the details). The most important thing is the order of the operations (for example, to get variables into the right scope, their defs have to appear in between the right push and pop scope operations):

1. push global scope ❶.
2. def variable x in current scope, ❶.
3. def method f in scope ❶ and push scope ❷.
4. def variable y.
5. push local scope ❸.
6. def variable i.
7. pop ❸ revealing ❷.
8. push local scope ❹.
9. def variable j.
10. pop ❹ revealing ❷.
11. pop function f scope ❷ revealing ❶.
12. def method g in scope ❶ and push scope ❺.
13. def variable i.
14. pop function g scope ❺ revealing ❶.
15. pop global scope ❶.

In the next section, we'll see how scope trees make it easy to find the symbols we've squirreled away.

6.3 Resolving Symbols

When we see a reference to symbol *x* in a program, our brain unconsciously looks for the closest definition. In other words, our brain tries to *resolve* (identify) which program entity it refers to. If we've only got one scope, resolving a symbol is easy. Either we find that symbol in that scope's Symbol dictionary, or we don't. In code, that looks like this:

```
myOnlyScope.resolve(«symbol-name»);
```

Method `resolve()` does nothing more than look up «*symbol-name*» in the scope's dictionary.

When there is more than one scope, though, resolving a symbol depends on the location of the symbol reference. In other words, the same symbol could refer to two different program entities depending on where the symbol appears in the source code. References in two different scopes see two different scope stacks. A reference's scope stack is the set of scopes on the path to the root of the scope tree. We call this stack the *semantic context*.

So, to resolve a symbol reference, we look for it in its semantic context, starting with the current scope. If `resolve()` doesn't find the symbol in the current scope, it asks the enclosing scope if it can find the symbol. `resolve()` recursively walks toward the root of the scope tree until it finds the symbol or runs out of scopes. The algorithm looks like this:

```
public Symbol resolve(String name) {
    Symbol s = members.get(name); // look in this scope
    if ( s!=null ) return s;      // return it if in this scope
    if ( enclosingScope != null ) { // have an enclosing scope?
        return enclosingScope.resolve(name); // check enclosing scope
    }
    return null; // not found in this scope or there's no scope above
}
```

The `enclosingScope` variable points to the scope above in the scope tree (it's a parent pointer). Don't let the recursion bother you. It's just the most convenient way to express the algorithm. The recursion in `resolve()` is essentially equivalent to a loop walking up the `enclosingScope` chain.

Here's the best part about this algorithm and scope tree combination. No matter how complicated our scope tree gets, we can always resolve symbols with the same bit of code:

```
currentScope.resolve(«symbol-name»);
```

The `enclosingScope` pointer tells `resolve()` exactly where to look next. Rather than making a clever algorithm that implements all of the look-up rules, we assemble a handy data structure that encodes them.

Let's see how method `resolve()` gracefully handles two important features of nested scopes: we can see symbols in outer scopes, and we can redefine symbols found in outer scopes. There is a little bit of variability between languages, but let's see how `resolve()` works on some C++ code:

```

❶ // start of global scope
int x;           // define variable x in global scope
int y;           // define variable y in global space
❷ void f() {     // define function f in global scope
    float x;     // redefine x as local variable, hiding outer x
    printf("%f", x); // x resolves to f's local
    printf("%d", y); // y resolves to global
❸    {int z;}    // local scope nested within f's local scope
    printf("%d", z); // z is no longer visible; static analysis ERROR!
}
```

There are three variable references in `f`, all in `printf` calls. The semantic context for the `x`, `y`, and `z` references in `f` is ❷❶. That means we should look up all symbol references starting in scope ❷. To resolve `x`, `resolve()` immediately finds it in ❷. `resolve()` doesn't find `y` in the current scope but does find it in enclosing scope ❶. We can't resolve `z` in the `printf`, though, because `z` is defined in ❸. That scope is not in the `z` reference's semantic context. `resolve()` returns null in this case, signaling an error.

As you can see, the structure of the scope tree reduces the complexity of symbol resolution to a simple walk up a tree (even when we tackle class inheritance in Chapter 7, *Managing Symbol Tables for Data Aggregates*, on page 170). Throughout the patterns that follow, we'll refer to this abstract reference resolution operation as `ref`. In total, that gives us four key abstract operations: `push`, `pop`, and `def` to construct scope trees and `ref` to resolve references in the scope tree. Armed with these operations and a good handle on symbol table management, we're ready to tackle our first two symbol table patterns. Here's a summary of when to apply the patterns:

Pattern

Pattern 16, *Symbol Table for Monolithic Scope*, on the next page

Pattern 17, *Symbol Table for Nested Scopes*, on page 161

When to Apply

This pattern applies to any language that defines symbols all in one scope.

If your language allows multiple scopes or even nested scopes, you'll need this pattern.

As we progress through the symbol table and semantic analysis patterns defined in the next few chapters, we should work with a common language for continuity purposes. Let's use a subset of C++ and call it "Cymbol."¹ We'll start with just variable definition syntax and trivial expressions. Then, we'll add functions, **structs**, and finally classes. To keep things simple, we'll avoid pointers until absolutely necessary and we'll disregard executable statements (they're not involved in symbol definition).

16

Symbol Table for Monolithic Scope



Purpose

This pattern builds a symbol table for a language with a single, flat scope.

This pattern is suitable for simple programming languages (without functions), configuration files, small graphics languages, and other small DSLs.

Discussion

The primary goal when building a symbol table is to construct a scope tree. In this case, the scope tree is pretty boring since it's a single node (the global scope). The following table indicates how to build the single scope by responding to input constructs. We technically don't need to push and pop scopes for this simple case, but we'll do so for consistency with the other patterns.

Upon	Action(s)
Start of file	push a GlobalScope. def BuiltInType objects for any built-in types such as int and float.
Declaration x	ref x's type (if any). def x in the current scope.
Reference x	ref x starting in the current scope.
End of file	pop the GlobalScope.

1. Not to be confused with the Cymbol language (registered trademark) in the AT&T Daytona project; <http://www.research.att.com/~daytona/>

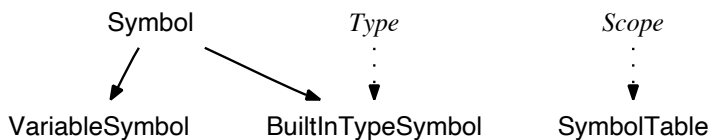
Let's apply those actions to the following Cymbol declarations:

[Download](#) `symtab/monolithic/t.cymbol`

```
int i = 9;
float j;
int k = i+2;
```

We'd execute the following sequence: push global scope, def int, def float, ref int, def i, ref float, def j, ref int, def k, ref i, pop global scope. The built-in type definitions are part of symbol table initialization; the parser doesn't trigger those actions because of an input statement. In simple terms, we're creating the global scope and then defining three variables: i, j, and k. To define a variable, we have to look up its type. We also have to look up variables referenced in initialization expressions.

To populate our symbol table, we need objects to represent variables and built-in types. We'll tag type symbols with interface *Type* and then make our symbol table object play the role of a scope using interface *Scope*. The class inheritance and interface implementation hierarchy looks like this:



The next section provides a complete Cymbol parser and implementations for these symbol table objects.

Implementation

Because expressions in this version of Cymbol cannot reference variables defined later in the file, we can define and properly reference all symbols in a single pass. We only need a parser and a few actions to demonstrate symbol table management.

For the Cymbol program shown earlier, we need to generate output that convinces us we've properly managed symbols. So, let's try to generate the following (which does not show the def operations for built-in types):

```
$ java Test < t.cymbol
line 1: ref int
line 1: def i
line 2: ref float
line 2: def j
line 3: ref int
line 3: ref to <i:int> // <i:int> means i has type int
```

```
line 3: def k
globals: {int=int, j=<j:float>, k=<k:int>, float=float, i=<i:int>}
$
```

As we encounter variable definitions, we'll add them to the symbol table and print a message. As we find variable and type references, we'll resolve them and also print a message. Right before terminating, we'll print out the symbols in the symbol table's only scope.

To get started, let's define symbols and then move on to tracking those symbols. Our generic Symbol object has a name and type properties:

[Download](#) symtab/monolithic/Symbol.java

```
public class Symbol { // A generic programming language symbol
    String name;      // All symbols at least have a name
    Type type;
    public Symbol(String name) { this.name = name; }
    public Symbol(String name, Type type) {this(name); this.type = type;}
    public String getName() { return name; }
    public String toString() {
        if ( type!=null ) return '<' + getName() + ":" + type + '>';
        return getName();
    }
}
```

This version of Cymbol only has two kinds of symbols: variables and built-in types:

[Download](#) symtab/monolithic/VariableSymbol.java

```
/** Represents a variable definition (name,type) in symbol table */
public class VariableSymbol extends Symbol {
    public VariableSymbol(String name, Type type) { super(name, type); }
}
```

[Download](#) symtab/monolithic/BuiltInTypeSymbol.java

```
/** A symbol for built in types such int, float primitive types */
public class BuiltInTypeSymbol extends Symbol implements Type {
    public BuiltInTypeSymbol(String name) { super(name); }
}
```

Now we need a dictionary to hold those symbols (it's a single-node tree or a scope stack of depth one). Let's put it in a SymbolTable object. Because it holds the sole dictionary (field symbols), we might as well make SymbolTable itself represent the Scope.

Download [symtab/monolithic/SymbolTable.java](#)

```
import java.util.*;
public class SymbolTable implements Scope { // single-scope symtab
    Map<String, Symbol> symbols = new HashMap<String, Symbol>();
    public SymbolTable() { initTypeSystem(); }
    protected void initTypeSystem() {
        define(new BuiltInTypeSymbol("int"));
        define(new BuiltInTypeSymbol("float"));
    }
    // Satisfy Scope interface
    public String getScopeName() { return "global"; }
    public Scope getEnclosingScope() { return null; }
    public void define(Symbol sym) { symbols.put(sym.name, sym); }
    public Symbol resolve(String name) { return symbols.get(name); }

    public String toString() { return getScopeName()+":"+symbols; }
}
```

At this point, we've got symbols and a symbol table. Let's give them a workout by creating a parser for Cymbol. Cymbol.g defines the Cymbol language, and we're going to insert actions that create symbols, sticks them in the symbol table, and then resolves them.

The details of the grammar aren't important. The take-away is that rules recognizing definitions create Symbol objects and call `define()`. Rules referencing identifiers invoke `resolve()` to look them up. Keep in mind that actions execute according to location in the grammar. For example, actions at the end of an alternative execute after the parser matches that construct. Actions use terms like `$ID` to access the token matched by an `ID` token.

First, we need to add a parameter to the start symbol, **compilationUnit**, so that we can pass in a symbol table:

Download [symtab/monolithic/Cymbol.g](#)

```
grammar Cymbol; // my grammar is called Cymbol
// define a SymbolTable field in generated parser
@members {SymbolTable symtab;}
compilationUnit[SymbolTable symtab] // pass symbol table to start rule
@init {this.symtab = symtab;} // set the parser's field
:   varDeclaration+ // recognize at least one variable declaration
;
```

SymbolTable defines two built-in types that we can look up during variable definitions. Rule **type** looks up the type name and returns it as a Type symbol.

[Download](#) symtab/monolithic/Cymbol.g

```
type returns [Type tsym]
@after { // $start is the first tree node matched by this rule
    System.out.println("line "+$start.getLine()+": ref "+$tsym.getName());
}
:   'float' {$tsym = (Type)symtab.resolve("float");}
|   'int'   {$tsym = (Type)symtab.resolve("int");}
;
```

Then, rule **varDeclaration** can create a `VariableSymbol` with that `Type`:

[Download](#) symtab/monolithic/Cymbol.g

```
varDeclaration
:   type ID ('=' expression)? ';' // E.g., "int i = 2;", "int i;"
    {
        System.out.println("line "+$ID.getLine()+": def "+$ID.text);
        VariableSymbol vs = new VariableSymbol($ID.text,$type.tsym);
        symtab.define(vs);
    }
;
```

Expression `$type.tsym` evaluates to the return value from invoking rule **type**.

And, finally, we have to resolve symbol references. The only place in the grammar where we reference variables is in declaration initialization expressions:

[Download](#) symtab/monolithic/Cymbol.g

```
primary
:   ID // reference variable in an expression
    {System.out.println("line "+$ID.getLine()+": ref to "+
        symtab.resolve($ID.text));}
|   INT
|   '(' expression ')'
;
```

To exercise the symbol table management in our Cymbol grammar, we can use the following test rig:

[Download](#) symtab/monolithic/Test.java

```
CharStream input = null; // read from filename or stdin
if ( args.length>0 ) input = new ANTLRFileStream(args[0]);
else input = new ANTLRInputStream(System.in);
CymbolLexer lex = new CymbolLexer(input); // create lexer
CommonTokenStream tokens = new CommonTokenStream(lex);
CymbolParser p = new CymbolParser(tokens); // create parser
SymbolTable symtab = new SymbolTable(); // create symbol table
p.compilationUnit(symtab); // launch parser
System.out.println("globals: "+symtab.symbols);
```

Here's how to build the rig, run ANTLR on the Cymbol.g grammar, compile, and run sample input t.cymbol into Test:

```
$ java org.antlr.Tool Cymbol.g
$ javac *.java
$ java Test < t.cymbol
line 1: ref int
...
```

This pattern is a bare-bones symbol table manager. Although it's sufficient for simple DSLs, it mostly serves as a good learning tool for building more realistic symbol tables.

Related Patterns

The next pattern, Pattern 17, *Symbol Table for Nested Scopes*, adds functions and nested scopes to our Cymbol C++ subset.

17

Symbol Table for Nested Scopes



Purpose

This pattern tracks symbols and builds a scope tree for languages with multiple, possibly nested scopes.

Programming language functions are a good example of nested scopes. Each function has its own scope that is nested within a global or class scope. Some languages even support nested function definitions or multiple local scopes. Many DSLs have nested scopes as well. The DOT² graphics language, for example, has subgraphs within graphs. This pattern handles them all gracefully.

Discussion

To discuss nested scopes, let's add functions to our Cymbol C++ subset. That means we'll need a function (method) Symbol and scopes for globals, parameters, and local variables. Let's take a look at some sample input and identify the various scopes using numbered icons.

2. <http://www.graphviz.org>

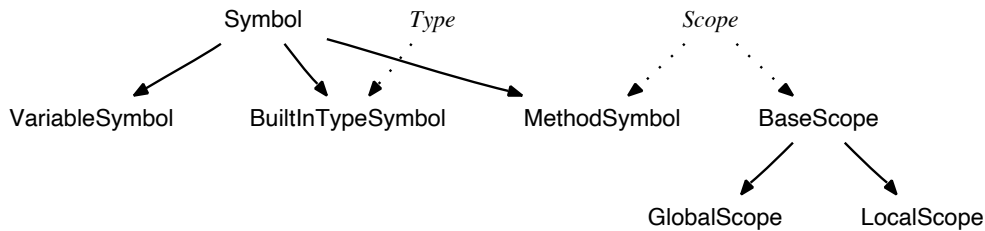
[Download](#) symtab/nested/t.cymbol

```

① // start of global scope
   int i = 9;
② float f(int x, float y)
③ {
    float i;
④    { float z = x+y; i = z; }
⑤    { float z = i+1; i = z; }
    return i;
⑥ }
⑦ void g()
   {
    f(i,2);
   }

```

We'll need two more Symbol subclasses to handle functions and local scopes. The following class inheritance and interface implementation hierarchy identifies all the symbol table objects:



MethodSymbol is both a symbol and a scope. Methods have two scopes actually: one for the parameters (the MethodSymbol itself) and one for its local variables (a LocalScope whose parent is the MethodSymbol).

Now that we know the class names of our symbol table objects, we can make more explicit scope trees than we could before. Previous scope trees (such as Figure 6.1, on page 152) only really showed tree structure. We can see the scope tree for t.cymbol, mentioned earlier, in Figure 6.2, on the next page; it identifies some key object properties. The goal of this pattern is to construct such scope trees.

In Figure 6.3, on the following page, we can see the rules laid out for building a scope tree from nested scopes and resolving symbols in the correct semantic context. The sequence of actions for t.cymbol starts like this: push global scope, def int, def float, ref int, def i, ref float, def f, push f, ref int, def x, ref float, def y, push local scope of f, ref float, def i, and so on.

Notice that we're not making a distinction between global variables, parameters, and local variables in terms of symbol table logic; they are

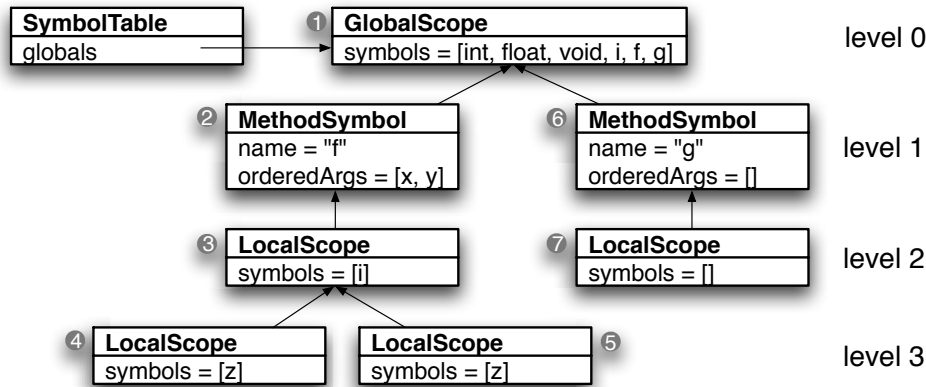


Figure 6.2: Scope tree for global variable and two functions

Upon	Action(s)
Start of file	push a GlobalScope. def BuiltInType objects for int, float, void.
Variable declaration x	ref x's type. def x in the current scope.
Method declaration f	ref f's return type. def f in the current scope and push it as the current scope.
{	push a LocalScope as the new current scope.
}	pop, revealing previous scope as current scope.
End of method	pop the MethodSymbol scope (the parameters).
Reference x	ref x starting in the current scope. If not found, look in the immediately enclosing scope (if any).
End of file	pop the GlobalScope.

Figure 6.3: Rules for building a scope tree for nested scopes and populating it with symbols

all `VariableSymbol` objects. The only difference between them is the scope in which we define them. For example, we define the parameters for `f` by executing `def` operations after pushing `f`'s method scope and before pushing its local scope.

Now that we've got the big picture, let's see how to use these rules in practice to manage a symbol table for nested scopes.

Implementation

For our particular implementation, we're going to trigger scope tree construction actions while walking an AST created from the input source code. We could trigger actions directly in the parser as we did in Pattern 16, *Symbol Table for Monolithic Scope*, on page 156, but working on an AST is more flexible. In Pattern 19, *Symbol Table for Classes*, on page 182, we'll need to make multiple passes over the input so we might as well get used to them now.

Starting from the sample implementation source code in the previous pattern, we need to do the following to add functions and nested scopes:

1. *Augment syntax to support functions.* We need to augment grammar `Cymbol.g` to support function definitions, function calls, return statements, and nested code blocks.
2. *Build an AST.* We will use AST construction rules in the parser grammar.
3. *Define new symbol table objects.* To support scopes, we'll need an abstract `BaseScope` and two concrete implementations: `GlobalScope` and `LocalScope`. `BaseScope` implements interface `Scope`. Finally, we'll need `MethodSymbol` that plays double duty as a `Symbol` and a `Scope` (for parameters).
4. *Walk the AST to populate the symbol table and resolve variable and method references.* Using tree pattern matching rules, we'll trigger actions according to Figure 6.3, on the previous page.

In this pattern and Pattern 18, *Symbol Table for Data Aggregates*, on page 176, we'll combine symbol definition and resolution into a single tree walk. In Pattern 19, *Symbol Table for Classes*, on page 182, we'll separate them into two tree-walking phases to support forward references. (Methods can call methods defined later in the class.)

Before getting into all the gory details, let's assume for the moment that we've got everything set up to build ASTs from source code. Because

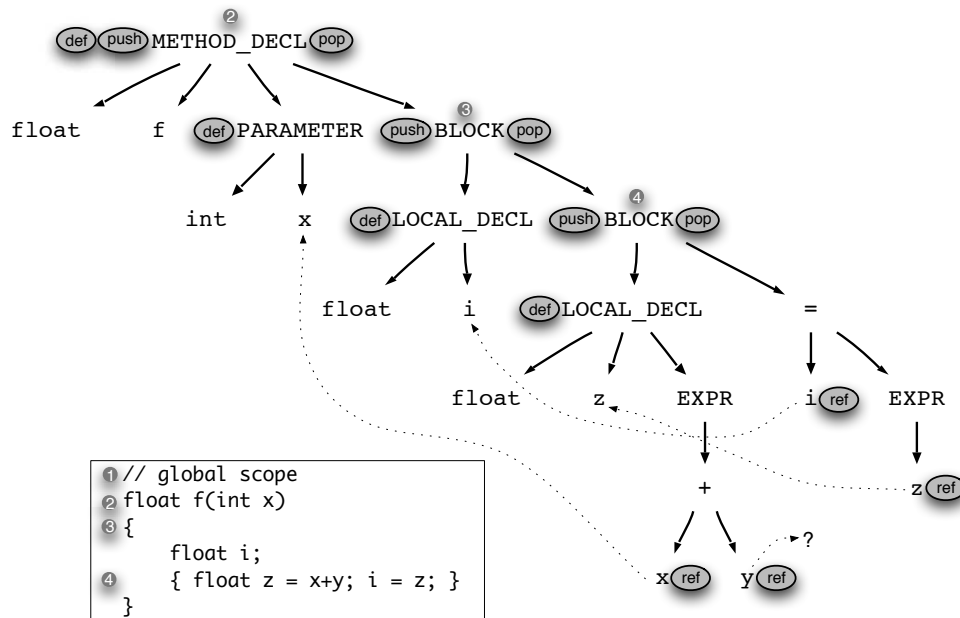


Figure 6.4: AST for function with nested scopes

all the magic happens with the AST, let's look at the AST of a simple function and annotate it with action execution and scope icons. In Figure 6.4, we can see where and when in the AST we need to execute push, pop, def, and ref actions. (Actually, we're ignoring built-in type lookups to reduce clutter.) Those actions appearing on the left of an AST node trigger as we discover nodes (on the way down) and those appearing on the right trigger as we finish nodes (on the way up). The dotted lines point from variable reference sites to declaration sites in the AST (note that the `y` reference has no corresponding definition).

To create a scope tree then, all we have to do is a depth-first walk of the AST, executing actions in the pre- and/or postorder position. We push as we descend and pop as we ascend. When we see a symbol, we define or resolve it in the current scope.

On to our implementation—the following sections summarize the software in the source code directory for this pattern.

Adding Function Syntax and Building ASTs

To add functions to Cymbol, we'll add rules **methodDeclaration** and **formalParameters** to Cymbol.g. Beyond the syntax, we have to add AST construction rules. For example, here is the previous **varDeclaration** rule augmented with an AST construction rule:

[Download](#) `symtab/nested/Cymbol.g`

```
varDeclaration
:   type ID ('=' expression)? ';' -> ^(VAR_DECL type ID expression?)
;
```

The rule now yields **VAR_DECL**-rooted subtrees with a type, an identifier, and an optional initialization expression as children. Method declarations yield **METHOD_DECL**-rooted subtrees:

[Download](#) `symtab/nested/Cymbol.g`

```
methodDeclaration
:   type ID '(' formalParameters? ')' block
    -> ^(METHOD_DECL type ID formalParameters? block)
;
```

Parsing and AST construction are not the primary focus here, so let's move on to building the scope tree.

Building the Scope Tree

The rules we defined earlier for building scope trees (in Figure 6.3, on page 163) are of the form “upon an input construct *foo*, execute action *bar*.” The most direct implementation is a set of tree pattern-action pairs. For example, at the starting { of a code block, we need to push a new scope. At the ending } of a block, we need to pop that same scope off. The phrases “at the start” and “at the end” are synonymous with “on the way down” and “on the way up” the AST. Here are the tree pattern matching rules that handle pushing and popping local scopes:

[Download](#) `symtab/nested/DefRef.g`

```
enterBlock
:   BLOCK {currentScope = new LocalScope(currentScope);} // push scope
;
exitBlock
:   BLOCK
    {
      System.out.println("locals: "+currentScope);
      currentScope = currentScope.getEnclosingScope(); // pop scope
    }
;
```

The rules match the same **BLOCK** node but do different actions (push vs. pop) depending on whether we are discovering or finishing that node. We control which rules to execute on the way down vs. on the way up by listing them in the special **topdown** or **bottomup** rules:

```
topdown : enterBlock | enterMethod | ... ;
bottomup : exitBlock | exitMethod | ... ;
```

To manage method scopes, we create a method symbol and push it on the way down. On the way back up, we don't need to look inside the method subtree, so we match just the **METHOD_DECL** root. Here are the rules that deal with method definition subtrees:

[Download](#) symtab/nested/DefRef.g

```
enterMethod // match method subtree with 0-or-more args
:  ^(METHOD_DECL type ID .*)
{
    System.out.println("line "+$ID.getLine()+": def method "+
                        $ID.text);
    Type retType = $type.tsym; // rule type returns a Type symbol
    MethodSymbol ms = new MethodSymbol($ID.text,retType,
                                       currentScope);
    currentScope.define(ms); // def method in globals
    currentScope = ms;      // set current scope to method scope
}
;
exitMethod
:  METHOD_DECL
{
    System.out.println("args: "+currentScope);
    currentScope = currentScope.getEnclosingScope(); // pop arg scope
}
;
```

You might be wondering where we create the local scopes for methods. Rule **enterBlock** creates them automatically for us since method ASTs contain **BLOCK** subtrees (see Figure 6.4, on page 165).

Populating the Symbol Table

Now that we've got the proper scope tree structure, we just have to fill the scopes with symbols as we walk the AST. Fortunately, all definitions do the same thing: create the appropriate Symbol and then call `Scope.define()`.

Here is the rule to define any kind of variable:

[Download](#) symtab/nested/DefRef.g

```
varDeclaration // global, parameter, or local variable
:   ^((ARG_DECL|VAR_DECL) type ID .?)
    {
        System.out.println("line "+$ID.getLine()+": def "+$ID.text);
        VariableSymbol vs = new VariableSymbol($ID.text,$type.tsym);
        currentScope.define(vs);
    }
;
```

Field `currentScope` is always set to the current scope. Our tree pattern matcher always defines symbols within the current scope.

Let's do the opposite now and find those definitions in the scope tree.

Resolving Variable and Method References

In Chapter 8, *Enforcing Static Typing Rules*, on page 196, we'll be looking up symbols all the time to do type checking and so on. To give you a taste, here is a rule that looks up all identifiers that appear in expressions:

[Download](#) symtab/nested/DefRef.g

```
idref
:   {$start.hasAncestor(EXPR)}? ID
    {
        Symbol s = currentScope.resolve($ID.text);
        System.out.println("line "+$ID.getLine()+": ref "+s);
    }
;
```

The `hasAncestor()` predicate permits rule **idref** to match an **ID** only when it sits somewhere under an **EXPR** node.

Let's give it the old smoke test. The test rig in the source code directory builds an AST from the source code and then walks the AST using ANTLR's built-in `downup()` strategy:

[Download](#) symtab/nested/Test.java

```
CommonTree t = (CommonTree)r.getTree(); // get tree result from parser
CommonTreeNodeStream nodes = new CommonTreeNodeStream(t);
nodes.setTokenStream(tokens);
SymbolTable symtab = new SymbolTable(); // make global scope, types
DefRef def = new DefRef(nodes, symtab); // use custom constructor
def.downup(t); // trigger symtab actions upon certain subtrees
System.out.println("globals: "+symtab.globals);
```

To build all this, run ANTLR on the grammars and compile:

```
$ java org.antlr.Tool Cymbol.g DefRef.g
$ javac *.java
$
```

Here is what the output looks like when we run the test rig against the Cymbol source code in Figure 6.4, on page 165:

```
$ java Test < t2.cymbol
line 2: def method f
line 2: def x
line 4: def i
line 5: def z
line 5: ref <x:int>
line 5: ref null           // y is not defined, resolves to null
line 5: ref <z:float>
line 5: assign to <i:float>
locals: [z]
locals: [i]
args: method<f:float>:[<x:int>]
globals: [int, float, void, f]
$
```

Related Patterns

This pattern serves as the foundation for Pattern 18, *Symbol Table for Data Aggregates*, on page 176 and Pattern 19, *Symbol Table for Classes*, on page 182.

Up Next

Now that you're comfortable with nested scopes, you could build most of the symbol table infrastructure to track symbols for C. The only involved concept we haven't covered is the **struct** data aggregate. We'll remedy that in the next chapter.